AI ASSISTED CODING

LAB ASSIGNMENT-14.5

Name: C. Sai vamshi

HT No: 2303A52238

Batch: 35

Lab 14 – AI Assisted HTML / CSS / JavaScript Generation

Lab Objectives

- **Use AI to assist in designing and implementing fundamental data structures in Python**
- **Learn prompt-based structure creation, optimization, and documentation**
- **Improve understanding of Lists, Stacks, Queues, Linked Lists, Graphs, and Hash Tables**
- **Enhance readability and performance with AI-generated suggestions**

Task 1 – Image Gallery Website

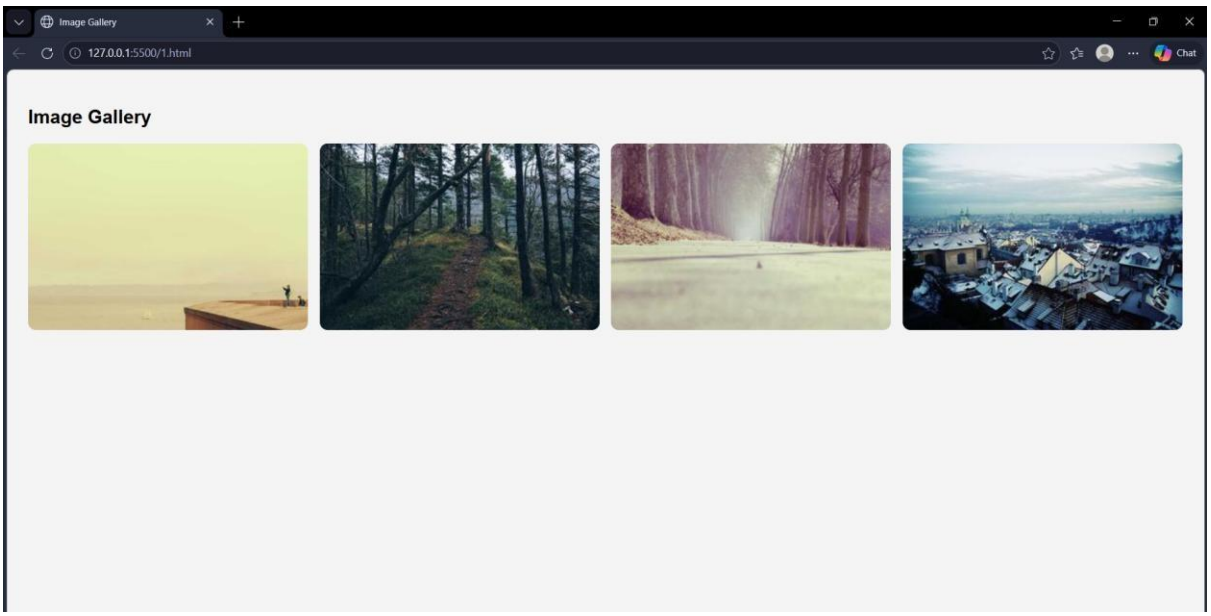
Prompt Given to AI

Generate a responsive image gallery website using HTML, CSS, and JavaScript.

Requirements:
• Display images in a grid layout using CSS Grid.
• Add hover zoom effect on images.
• Ensure the gallery is mobile responsive.
• Use clean modern UI styling.
• Provide complete code in a single HTML file.

```html
1.html > html
<!DOCTYPE html>
<html>
<head>
<title>Image Gallery</title>

<style>
body{
font-family: Arial;
background: #f4f4f4;
padding:20px;
}

.gallery{
display:grid;
grid-template-columns:repeat(auto-fit,minmax(200px,1fr));
gap:15px;
}

.gallery img{
width:100%;
border-radius:10px;
transition:transform 0.3s;
}

.gallery img:hover{
transform:scale(1.1);
}
</style>

</head>

<body>

<h2>Image Gallery</h2>

<div class="gallery">
<img src="https://picsum.photos/300/200?1">
<img src="https://picsum.photos/300/200?2">
<img src="https://picsum.photos/300/200?3">
<img src="https://picsum.photos/300/200?4">
</div>

</body>
</html>
```

Click to close server

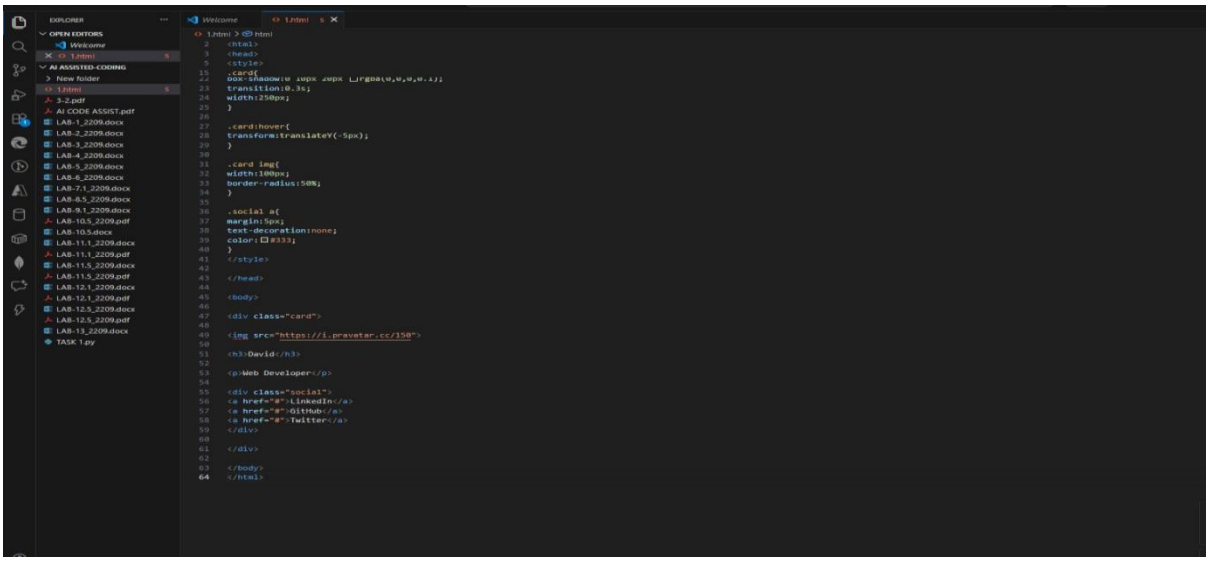


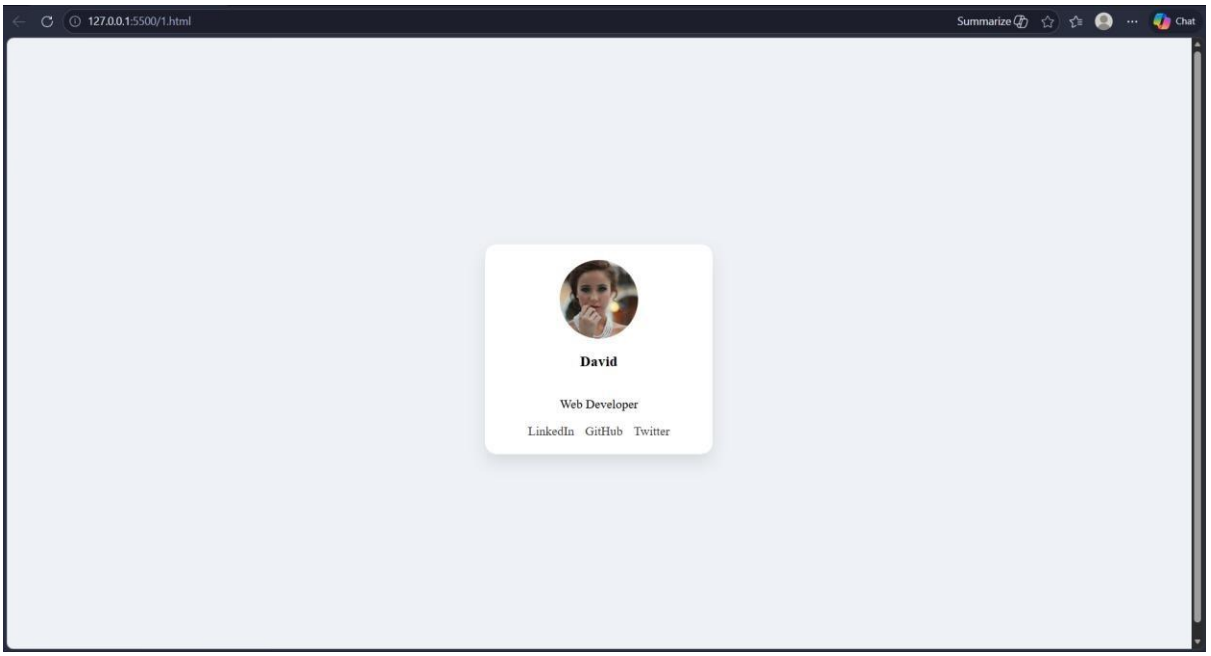
Task 2 – Profile Card Generator

Prompt Given to AI

Create a modern profile card using HTML and CSS.

Requirements:
- **Display profile image, name, role, and social media links.**
- **Use Flexbox for layout.**
- **Apply modern colors and fonts.**
- **Add hover animation and shadow effects.**
- **Provide the complete code.**





Task 3 – Dynamic Shopping Cart System

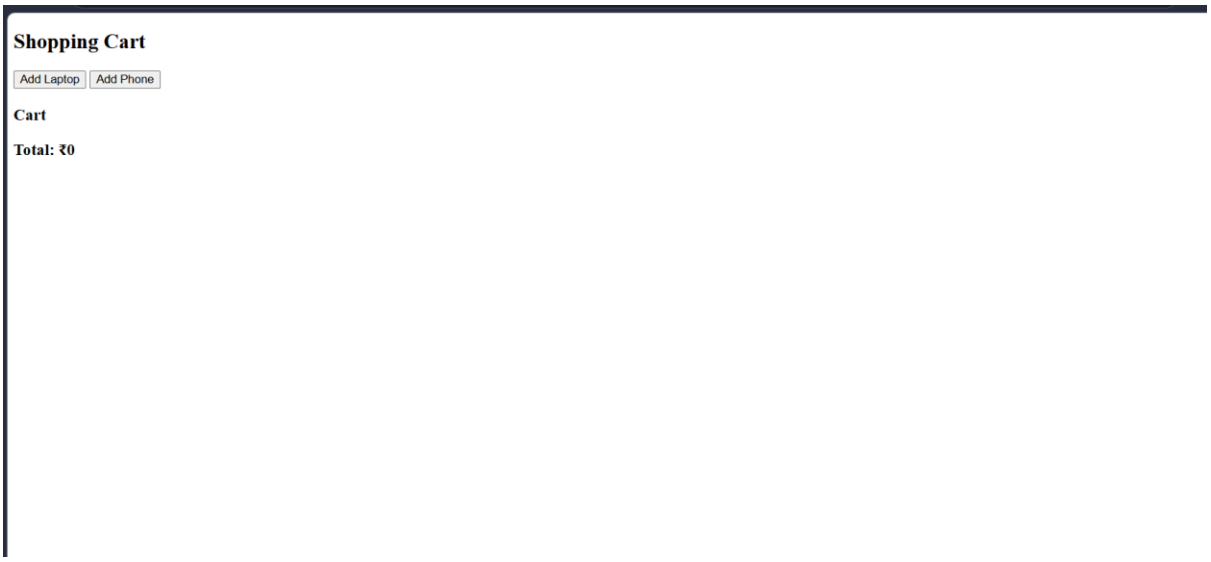
Prompt Given to AI

Create a dynamic shopping cart system using HTML, CSS, and JavaScript.

Requirements:
- **Add and remove products.**
- **Update product quantities.**
- **Display total price dynamically.**
- **Use JavaScript objects to manage cart state.**
- **Store cart data in localStorage.**
- **Provide complete working code.**

1.html > html

```html
<html>
<body>
<script>
function addProduct(name,price){

cart.push({name,price})

localStorage.setItem("cart",JSON.stringify(cart))

displayCart()

}

function displayCart(){

let cartList=document.getElementById("cart")

cartList.innerHTML=""

let total=0

cart.forEach((item,index)=>{

let li=document.createElement("li")

li.innerHTML=item.name+" - "+item.price+
` <button onclick="removeItem(${index})">Remove</button>`

cartList.appendChild(li)

total+=item.price

})

document.getElementById("total").innerText=total

}

function removeItem(index){

cart.splice(index,1)

localStorage.setItem("cart",JSON.stringify(cart))

displayCart()

}

displayCart()

</script>

</body>
</html>
```



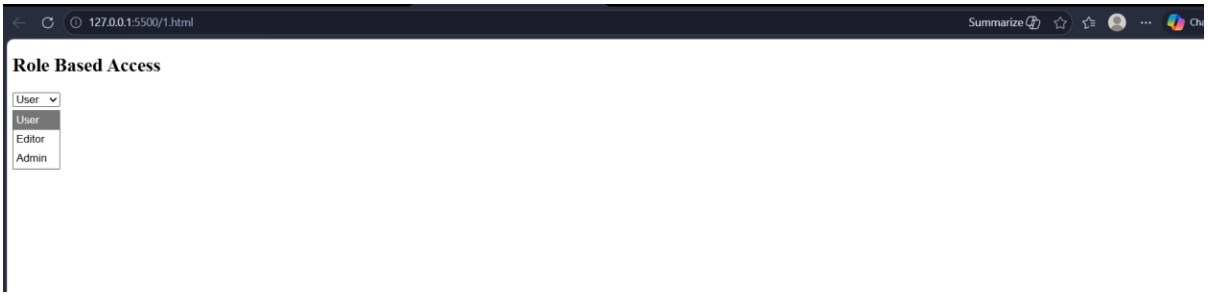
Task 4 – Role Based Access Control

Prompt Given to AI

Create a role-based web application using HTML and JavaScript.

Requirements:
• Support three roles: Admin, Editor, and User.
• Display different UI components based on selected role.
• Implement conditional UI rendering.
• Provide clean and simple code.

```html
1.html > html
<!DOCTYPE html>
<html>
<body>

<h2>Role Based Access</h2>

<select id="role" onchange="checkRole()">
<option value="user">User</option>
<option value="editor">Editor</option>
<option value="admin">Admin</option>
</select>

<div id="adminPanel">Admin Panel</div>
<div id="editorPanel">Editor Tools</div>

<script>

function checkRole(){

let role=document.getElementById("role").value

document.getElementById("adminPanel").style.display="none"
document.getElementById("editorPanel").style.display="none"

if(role=="admin"){
document.getElementById("adminPanel").style.display="block"
}

if(role=="editor"){
document.getElementById("editorPanel").style.display="block"
}

}

checkRole()

</script>

</body>
</html>
```



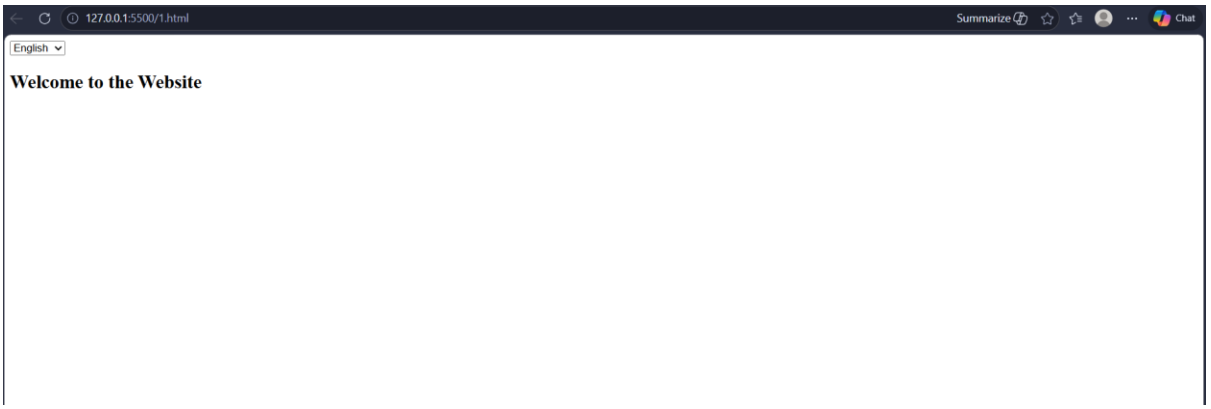
Task 5 – Multi Language Website

Prompt Given to AI

Create a multilingual website using HTML and JavaScript.

Requirements:
- **Allow users to switch between languages dynamically.**
- **Store language preference using localStorage.**
- **Use a translation object or JSON structure.**
- **Provide simple example with two languages.**

```html
<!DOCTYPE html>
<html>
<body>

<select onchange="changeLang(this.value)">
<option value="en">English</option>
<option value="es">Spanish</option>
</select>

<h2 id="title"></h2>

<script>

const translations={
en:{title:"Welcome to the Website"},
es:{title:"Bienvenido al sitio web"}
}

function changeLang(lang){

localStorage.setItem("lang",lang)

document.getElementById("title").innerText=
translations[lang].title

}

let lang=localStorage.getItem("lang") || "en"

changeLang(lang)

</script>

</body>
</html>
```



Task 6 – Online Examination System

Prompt Given to AI

Create a simple online examination interface using HTML, CSS, and JavaScript.

Requirements:
- **Display a question with options.**
- **Include a countdown timer.**
- **Automatically submit when time ends.**

• **Prevent multiple submissions.**
• **Show result after**

```
1.html > html
<html>
<body>

<input type="radio" name="q1" value="3">3
<input type="radio" name="q1" value="4">4

<br><br>

<button onclick="submitExam()">Submit</button>

<p id="result"></p>

<script>

let time=60

let examSubmitted=false

let timer=setInterval(()=>{

time--

document.getElementById("timer").innerText="Time: "+time

if(time<=0){
clearInterval(timer)
submitExam()
}

},1000)

function submitExam(){

if(examSubmitted) return

examSubmitted=true

clearInterval(timer)

let answer=document.querySelector('input[name="q1"]:checked')

if(answer && answer.value=="4"){
document.getElementById("result").innerText="Correct"
}
else{
document.getElementById("result").innerText="Wrong"
}

}

</script>

</body>
</html>
```

submission.

Online Exam

Time: 38

1. What is 2 + 2?

○ 3 ◉ 4

Submit

Correct

Conclusion

AI-assisted coding tools help developers rapidly generate responsive UI layouts, optimize JavaScript logic, and implement modern web development practices. Using HTML, CSS Grid/Flexbox, and JavaScript, developers can build interactive and scalable web applications while improving usability, responsiveness, and performance.